

Среда передачи сообщений и полный пример программы управления банкоматом

В главе 4 я приводил пример отправки сообщений между потоками с использованием среды передачи сообщений. Там применялась простая реализация кода управления банкоматом. Приведу полный код этого примера, включая среду передачи сообщений.

В листинге B.1 показан код очереди сообщений. Список сообщений хранится в виде указателей на базовый класс. Конкретный тип сообщений обрабатывается классом шаблона, являющимся производным от этого базового класса. Помещение записи в очередь приводит к созданию соответствующего экземпляра класса-оболочки и сохранению указателя на него. Извлечение записи приводит к возвращению этого указателя. Поскольку класс `message_base` не имеет компонентных функций, извлекающему потоку, прежде чем он сможет обратиться к сохраненному сообщению, потребуется приведение указателя к подходящему типу указателя `wrapped_message<T>`.

Листинг B.1. Простая очередь сообщений

```
#include <mutex>
#include <condition_variable>
#include <queue>
#include <memory>
namespace messaging
{
    struct message_base ← Базовый класс записей
                        вашей очереди
    {
        virtual ~message_base()
        {}
    };
};
```

```

template<typename Msg>
struct wrapped_message:
    message_base
{
    Msg contents;
    explicit wrapped_message(Msg const& contents_):
        contents(contents_)
    {}
};
class queue
{
    std::mutex m;
    std::condition_variable c;
    std::queue<std::shared_ptr<message_base> > q;
public:
    template<typename T>
    void push(T const& msg)
    {
        std::lock_guard<std::mutex> lk(m);
        q.push(std::make_shared<wrapped_message<T> >(msg));
        c.notify_all();
    }
    std::shared_ptr<message_base> wait_and_pop()
    {
        std::unique_lock<std::mutex> lk(m);
        c.wait(lk,[&]{return !q.empty();});
        auto res=q.front();
        q.pop();
        return res;
    }
};

```

У каждого типа сообщения своя специализация

Ваша очередь сообщений

Во внутренней очереди сохраняются указатели на message_base

Заключение размещенного сообщения в оболочку и сохранение указателя

Блокировка до минимального заполнения очереди

Отправкой сообщений занимается экземпляр класса `sender`, показанный в листинге В.2. Он представляет собой тонкую оболочку вокруг очереди сообщений, позволяющую лишь помещать сообщения. При копировании экземпляров класса `sender` копируется указатель на очередь, а не сама очередь.

Листинг В.2. Класс `sender`

```

namespace messaging
{
    class sender
    {
        queue*q;
    public:
        sender():
            q(nullptr)
        {}
        explicit sender(queue*q_):
            q(q_)
        {}
        template<typename Message>

```

sender является оболочкой вокруг указателя на очередь

Изначально созданный экземпляр класса sender не имеет очереди

Разрешение создания очереди из указателя

```

void send(Message const& msg)
{
    if(q)
    {
        q->push(msg);
    }
};
}

```

← При отправке сообщение помещается в очередь

Получение сообщений устроено несколько сложнее. Приходится не только ждать сообщения из очереди, но также и проверять, соответствует ли его тип любому ожидаемому типу, и вызывать соответствующую функцию обработки. Все это начинается с класса `receiver`, показанного в листинге В.3.

Листинг В.3. Класс `receiver`

```

namespace messaging
{
    class receiver
    {
        queue q;
    public:
        operator sender()
        {
            return sender(&q);
        }
        dispatcher wait()
        {
            return dispatcher(&q);
        }
    };
}

```

← receiver владеет очередью

← Разрешение подразумеваемого преобразования в объект `sender`, ссылающийся на очередь

← Ожидание, пока очередь не создаст `dispatcher`

Если `sender` ссылается на очередь сообщений, то `receiver` владеет этой очередью. Можно получить объект `sender`, ссылающийся на очередь, путем использования подразумеваемого преобразования. Работа по диспетчеризации сообщений начинается с вызова функции `wait()`. При этом создается объект `dispatcher`, ссылающийся на очередь из объекта `receiver`. Класс `dispatcher` показан в листинге В.4, где можно увидеть, что работа выполнена в деструкторе. В данном случае работа состоит из ожидания сообщения и его диспетчеризации.

Листинг В.4. Класс `dispatcher`

```

namespace messaging
{
    class close_queue
    {};
    class dispatcher
    {
        queue* q;
        bool chained;
        dispatcher(dispatcher const&)=delete;
    };
}

```

← Сообщение о закрытии очереди

← Экземпляры `dispatcher` нельзя скопировать

```

dispatcher& operator=(dispatcher const&)=delete;
template<
    typename Dispatcher,
    typename Msg,
    typename Func>
friend class TemplateDispatcher;
void wait_and_dispatch()
{
    for(;;)
    {
        auto msg=q->wait_and_pop();
        dispatch(msg);
    }
}
bool dispatch(
    std::shared_ptr<message_base> const& msg)
{
    if(dynamic_cast<wrapped_message<close_queue>*>(msg.get()))
    {
        throw close_queue();
    }
    return false;
}
public:
dispatcher(dispatcher&& other):
{
    q(other.q),chained(other.chained)
    {
        other.chained=true;
    }
explicit dispatcher(queue* q_):
    q(q_),chained(false)
{
}
template<typename Message,typename Func>
TemplateDispatcher<dispatcher,Message,Func>
handle(Func&& f)
{
    return TemplateDispatcher<dispatcher,Message,Func>(
        q,this,std::forward<Func>(f));
}
~dispatcher() noexcept(false)
{
    if(!chained)
    {
        wait_and_dispatch();
    }
}
};
}

```

Разрешение экземплярам TemplateDispatcher обращаться к внутренним компонентам класса

Цикл, ожидание и диспетчеризация сообщений

dispatch() проверяет наличие сообщения close_queue и выдает исключение

Экземпляры dispatcher могут перемещаться

Источник не должен ждать сообщений

Обработка конкретного типа сообщения с помощью TemplateDispatcher

Деструктор может выдать исключение

Экземпляр dispatcher, возвращенный функцией wait(), будет тут же уничтожен, поскольку он временный и, как мы говорили, вся работа выполняется в деструкторе. Деструктор вызывает функцию wait_and_dispatch(), представляющую собой цикл ❶, в котором ожидается сообщение, передаваемое функции dispatch(). Сама

функция `dispatch()` ❷ предельно проста, она проверяет, не поступило ли сообщение о закрытии очереди `close_queue`, и выдает исключение, если такое сообщение поступило; в противном случае она возвращает `false`, чтобы показать, что это сообщение не было обработано. Из-за этого исключения `close_queue` деструктор помечен как `noexcept(false)`; без этой аннотации спецификацией исключения по умолчанию для деструктора стала бы `noexcept(true)` ❹, показывающая невозможность выдачи исключения, и тогда исключение `close_queue` привело бы к прекращению выполнения программы.

Но сама по себе функция `wait()` вызывается нечасто; большую часть времени приходится тратить на обработку сообщения. И здесь на первый план выходит компонентная функция `handle()` ❸. Это шаблон, и тип сообщения не выводится, поэтому нужно указать тип обрабатываемого сообщения и передать функцию (или вызываемый объект) для его обработки. Сама функция `handle()` для обработки сообщений указанного типа передает очередь, текущий объект `dispatcher` и функцию-обработчик новому экземпляру шаблона класса `TemplateDispatcher`, код которого показан в листинге В.5. Именно поэтому значение `chained` проверяется в деструкторе до ожидания сообщений. Тем самым предотвращается не только ожидание сообщений перемещенными откуда-то объектами, но и разрешается переложить ответственность за ожидание на ваш новый экземпляр `TemplateDispatcher`.

Листинг В.5. Шаблон класса `TemplateDispatcher`

```
namespace messaging
{
    template<typename PreviousDispatcher, typename Msg, typename Func>
    class TemplateDispatcher
    {
    public:
        queue* q;
        PreviousDispatcher* prev;
        Func f;
        bool chained;
        TemplateDispatcher(TemplateDispatcher const&)=delete;
        TemplateDispatcher& operator=(TemplateDispatcher const&)=delete;
        template<typename Dispatcher, typename OtherMsg, typename OtherFunc>
        friend class TemplateDispatcher;
        void wait_and_dispatch()
        {
            for(;;)
            {
                auto msg=q->wait_and_pop();
                if(dispatch(msg))
                    break;
            }
        }
        bool dispatch(std::shared_ptr<message_base> const& msg)
        {
            if(wrapped_message<Msg>* wrapper=
                dynamic_cast<wrapped_message<Msg>*>(msg.get()))
            {
                f(wrapper->contents);
                return true;
            }
        }
    };
}
```

Экземпляры `TemplateDispatcher` дружат друг с другом

❶ Если сообщение обрабатывается, нужно выйти из цикла

❷ Проверка типа сообщения и вызов функции

```

    }
    else
    {
        return prev->dispatch(msg);
    }
}
public:
    TemplateDispatcher(TemplateDispatcher&& other):
        q(other.q),prev(other.prev),f(std::move(other.f)),
        chained(other.chained)
    {
        other.chained=true;
    }
    TemplateDispatcher(queue* q_,PreviousDispatcher* prev_,Func&& f_):
        q(q_),prev(prev_),f(std::forward<Func>(f_)),chained(false)
    {
        prev_->chained=true;
    }
    template<typename OtherMsg,typename OtherFunc>
    TemplateDispatcher<TemplateDispatcher,OtherMsg,OtherFunc>
    handle(OtherFunc&& of)
    {
        return TemplateDispatcher<
            TemplateDispatcher,OtherMsg,OtherFunc>(
                q,this,std::forward<OtherFunc>(of));
    }
    ~TemplateDispatcher() noexcept(false)
    {
        if(!chained)
        {
            wait_and_dispatch();
        }
    }
};
}

```

Привязка к предыдущему диспетчеру

Дополнительные обработчики могут быть выстроены в цепочку

Деструктор снова помечен как noexcept(false)

Шаблон класса `TemplateDispatcher<>` смоделирован по образцу класса `dispatcher` и почти идентичен этому классу. В частности, деструктор по-прежнему вызывает функцию `wait_and_dispatch()`, чтобы дождаться сообщения.

Поскольку, если сообщение обрабатывается, исключение не выдается, теперь нужно проверить, обработано ли сообщение в вашем цикле сообщений ❶. Обработка сообщений прекращается, как только сообщение успешно обработано, чтобы в очередной раз можно было ожидать другого набора сообщений. Если получено совпадение с указанным типом сообщения, то вместо выдачи исключения вызывается предоставленная функция ❷ (хотя функция-обработчик сама может выдать исключение). Если совпадение не получено, выполняется переход по цепочке к предыдущему диспетчеру ❸. В первом экземпляре это будет объект `dispatcher`, но если выстроить цепочку из вызовов `handle()` ❹, позволяя обрабатывать несколько типов сообщений, это может быть предыдущий экземпляр `TemplateDispatcher<>`, который в случае несовпадения типа сообщения, в свою очередь, обратится по цепочке к предыдущему обработчику. Поскольку любой из обработчиков может выдать

исключение (не исключая обработчик диспетчера по умолчанию, предназначенного для обработки сообщений `close_queue`). Деструктор должен быть еще раз помечен как `noexcept(false)` ❸.

Эта простая среда позволяет помещать в очередь сообщение любого типа, а затем на стороне получателя выбирать путем проверки на совпадение сообщения, подлежащие обработке. Она также позволяет раздавать ссылку на очередь для помещения в нее сообщений, сохраняя закрытый характер получающей стороны.

Для завершения примера из главы 4 в листинге В.6 показан код сообщений, в листингах В.7, В.8 и В.9 — код различных конечных автоматов, а в листинге В.10 — управляющий код.

Листинг В.6. Сообщения банкомата

```
struct withdraw
{
    std::string account;
    unsigned amount;
    mutable messaging::sender atm_queue;
    withdraw(std::string const& account_,
             unsigned amount_,
             messaging::sender atm_queue_):
        account(account_), amount(amount_),
        atm_queue(atm_queue_)
    {}
};
struct withdraw_ok
{};
struct withdraw_denied
{};
struct cancel_withdrawal
{
    std::string account;
    unsigned amount;
    cancel_withdrawal(std::string const& account_,
                     unsigned amount_):
        account(account_), amount(amount_)
    {}
};
struct withdrawal_processed
{
    std::string account;
    unsigned amount;
    withdrawal_processed(std::string const& account_,
                       unsigned amount_):
        account(account_), amount(amount_)
    {}
};
struct card_inserted
{
    std::string account;
    explicit card_inserted(std::string const& account_):
```

```
        account(account_)
    {}
};
struct digit_pressed
{
    char digit;
    explicit digit_pressed(char digit_):
        digit(digit_)
    {}
};
struct clear_last_pressed
{};
struct eject_card
{};
struct withdraw_pressed
{
    unsigned amount;
    explicit withdraw_pressed(unsigned amount_):
        amount(amount_)
    {}
};
struct cancel_pressed
{};
struct issue_money
{
    unsigned amount;
    issue_money(unsigned amount_):
        amount(amount_)
    {}
};
struct verify_pin
{
    std::string account;
    std::string pin;
    mutable messaging::sender atm_queue;
    verify_pin(std::string const& account_, std::string const& pin_,
        messaging::sender atm_queue_):
        account(account_), pin(pin_), atm_queue(atm_queue_)
    {}
};
struct pin_verified
{};
struct pin_incorrect
{};
struct display_enter_pin
{};
struct display_enter_card
{};
struct display_insufficient_funds
{};
struct display_withdrawal_cancelled
{};
```

```

struct display_pin_incorrect_message
{};
struct display_withdrawal_options
{};
struct get_balance
{
    std::string account;
    mutable messaging::sender atm_queue;
    get_balance(std::string const& account_, messaging::sender atm_queue_):
        account(account_), atm_queue(atm_queue_)
    {}
};
struct balance
{
    unsigned amount;

    explicit balance(unsigned amount_):
        amount(amount_)
    {}
};
struct display_balance
{
    unsigned amount;
    explicit display_balance(unsigned amount_):
        amount(amount_)
    {}
};
struct balance_pressed
{};

```

Листинг В.7. Конечный автомат банкомата

```

class atm
{
    messaging::receiver incoming;
    messaging::sender bank;
    messaging::sender interface_hardware;
    void (atm::*state)();
    std::string account;
    unsigned withdrawal_amount;
    std::string pin;
    void process_withdrawal()
    {
        incoming.wait()
        .handle<withdraw_ok>(<
            [&](withdraw_ok const& msg)
            {
                interface_hardware.send(
                    issue_money(withdrawal_amount));
                bank.send(
                    withdrawal_processed(account, withdrawal_amount));
                state=&atm::done_processing;
            }
        );
    }
};

```

```

    }
    )
    .handle<withdraw_denied>(
        [&](withdraw_denied const& msg)
        {
            interface hardware.send(display_insufficient_funds());
            state=&atm::done_processing;
        }
    )
    .handle<cancel_pressed>(
        [&](cancel_pressed const& msg)
        {
            bank.send(
                cancel_withdrawal(account, withdrawal_amount));
            interface hardware.send(
                display_withdrawal_cancelled());
            state=&atm::done_processing;
        }
    );
}
void process_balance()
{
    incoming.wait()
    .handle<balance>(
        [&](balance const& msg)
        {
            interface hardware.send(display_balance(msg.amount));
            state=&atm::wait_for_action;
        }
    )
    .handle<cancel_pressed>(
        [&](cancel_pressed const& msg)
        {
            state=&atm::done_processing;
        }
    );
}
void wait_for_action()
{
    interface hardware.send(display_withdrawal_options());
    incoming.wait()
    .handle<withdraw_pressed>(
        [&](withdraw_pressed const& msg)
        {
            withdrawal_amount=msg.amount;
            bank.send(withdraw(account, msg.amount, incoming));
            state=&atm::process_withdrawal;
        }
    )
    .handle<balance_pressed>(
        [&](balance_pressed const& msg)
        {

```

```

        bank.send(get_balance(account,incoming));
        state=&atm::process_balance;
    }
}
.handle<cancel_pressed>(
    [&](cancel_pressed const& msg)
    {
        state=&atm::done_processing;
    }
);
}
void verifying_pin()
{
    incoming.wait()
    .handle<pin_verified>(
        [&](pin_verified const& msg)
        {
            state=&atm::wait_for_action;
        }
    )
    .handle<pin_incorrect>(
        [&](pin_incorrect const& msg)
        {
            interfaceHardware.send(
                display_pin_incorrect_message());
            state=&atm::done_processing;
        }
    )
    .handle<cancel_pressed>(
        [&](cancel_pressed const& msg)
        {
            state=&atm::done_processing;
        }
    );
}
void getting_pin()
{
    incoming.wait()
    .handle<digit_pressed>(
        [&](digit_pressed const& msg)
        {
            unsigned const pin_length=4;
            pin+=msg.digit;
            if(pin.length()==pin_length)
            {
                bank.send(verify_pin(account,pin,incoming));
                state=&atm::verifying_pin;
            }
        }
    )
    .handle<clear_last_pressed>(
        [&](clear_last_pressed const& msg)

```

```

        {
            if(!pin.empty())
            {
                pin.pop_back();
            }
        }
    )
    .handle<cancel_pressed>(<
        [&](cancel_pressed const& msg)
        {
            state=&atm::done_processing;
        }
    );
}
void waiting_for_card()
{
    interface hardware.send(display_enter_card());
    incoming.wait()
    .handle<card_inserted>(<
        [&](card_inserted const& msg)
        {
            account=msg.account;
            pin="";
            interface hardware.send(display_enter_pin());
            state=&atm::getting_pin;
        }
    );
}
void done_processing()
{
    interface hardware.send(eject_card());
    state=&atm::waiting_for_card;
}
atm(atm const&)=delete;
atm& operator=(atm const&)=delete;
public:
    atm(messaging::sender bank_,
        messaging::sender interface_hardware_):
        bank(bank_), interface_hardware(interface_hardware_)
    {}
    void done()
    {
        get_sender().send(messaging::close_queue());
    }
    void run()
    {
        state=&atm::waiting_for_card;
        try
        {
            for(;;)
            {
                (this->*state)();
            }
        }
    }
}

```

```

    }
}
catch(messaging::close_queue const&)
{
}
}
messaging::sender get_sender()
{
    return incoming;
}
};

```

Листинг В.8. Конечный автомат банка

```

class bank_machine
{
    messaging::receiver incoming;
    unsigned balance;
public:
    bank_machine():
        balance(199)
    {}
    void done()
    {
        get_sender().send(messaging::close_queue());
    }
    void run()
    {
        try
        {
            for(;;)
            {
                incoming.wait()
                    .handle<verify_pin>(
                        [&](verify_pin const& msg)
                        {
                            if(msg.pin=="1937")
                            {
                                msg.atm_queue.send(pin_verified());
                            }
                            else
                            {
                                msg.atm_queue.send(pin_incorrect());
                            }
                        }
                    )
                .handle<withdraw>(
                    [&](withdraw const& msg)
                    {
                        if(balance>=msg.amount)
                        {
                            msg.atm_queue.send(withdraw_ok());
                        }
                    }
                );
            }
        }
    }
};

```

```

        balance -= msg.amount;
    }
    else
    {
        msg.atm_queue.send(withdraw_denied());
    }
}
)
.handle<get_balance>(
    [&](get_balance const& msg)
    {
        msg.atm_queue.send(::balance(balance));
    }
)
.handle<withdrawal_processed>(
    [&](withdrawal_processed const& msg)
    {
    }
)
.handle<cancel_withdrawal>(
    [&](cancel_withdrawal const& msg)
    {
    }
);
}
}
catch(messaging::close_queue const&)
{
}
}

messaging::sender get_sender()
{
    return incoming;
}
};

```

Листинг В.9. Конечный автомат интерфейса пользователя

```

class interface_machine
{
    messaging::receiver incoming;
public:
    void done()
    {
        get_sender().send(messaging::close_queue());
    }
    void run()
    {
        try
        {
            for(;;)

```

```

{
    incoming.wait()
    .handle<issue_money>(
        [&](issue_money const& msg)
        {
            {
                std::lock_guard<std::mutex> lk(iom);
                std::cout<<"Issuing "
                    <<msg.amount<<std::endl;
            }
        }
    )
    .handle<display_insufficient_funds>(
        [&](display_insufficient_funds const& msg)
        {
            {
                std::lock_guard<std::mutex> lk(iom);
                std::cout<<"Insufficient funds"<<std::endl;
            }
        }
    )
    .handle<display_enter_pin>(
        [&](display_enter_pin const& msg)
        {
            {
                std::lock_guard<std::mutex> lk(iom);
                std::cout
                    <<"Please enter your PIN (0-9)"
                    <<std::endl;
            }
        }
    )
    .handle<display_enter_card>(
        [&](display_enter_card const& msg)
        {
            {
                std::lock_guard<std::mutex> lk(iom);
                std::cout<<"Please enter your card (I)"
                    <<std::endl;
            }
        }
    )
    .handle<display_balance>(
        [&](display_balance const& msg)
        {
            {
                std::lock_guard<std::mutex> lk(iom);
                std::cout
                    <<"The balance of your account is "
                    <<msg.amount<<std::endl;
            }
        }
    )
}

```

```

    }
    .handle<display_withdrawal_options>(
        [&](display_withdrawal_options const& msg)
        {
            {
                std::lock_guard<std::mutex> lk(iom);
                std::cout<<"Withdraw 50? (w)"<<std::endl;
                std::cout<<"Display Balance? (b)"
                    <<std::endl;
                std::cout<<"Cancel? (c)"<<std::endl;
            }
        }
    )
    .handle<display_withdrawal_cancelled>(
        [&](display_withdrawal_cancelled const& msg)
        {
            {
                std::lock_guard<std::mutex> lk(iom);
                std::cout<<"Withdrawal cancelled"
                    <<std::endl;
            }
        }
    )
    .handle<display_pin_incorrect_message>(
        [&](display_pin_incorrect_message const& msg)
        {
            {
                std::lock_guard<std::mutex> lk(iom);
                std::cout<<"PIN incorrect"<<std::endl;
            }
        }
    )
    .handle<eject_card>(
        [&](eject_card const& msg)
        {
            {
                std::lock_guard<std::mutex> lk(iom);
                std::cout<<"Ejecting card"<<std::endl;
            }
        }
    );
}

}
catch(messaging::close_queue&)
{
}
}
messaging::sender get_sender()
{
    return incoming;
}
};

```

Листинг В.10. Управляющий код

```

int main()
{
    bank_machine bank;
    interface_machine interface hardware;
    atm machine(bank.get_sender(), interface hardware.get_sender());
    std::thread bank_thread(&bank_machine::run, &bank);
    std::thread if_thread(&interface_machine::run, &interface hardware);
    std::thread atm_thread(&atm::run, &machine);
    messaging::sender atmqueue(machine.get_sender());
    bool quit_pressed=false;
    while(!quit_pressed)
    {
        char c=getchar();
        switch(c)
        {
            case '0':
            case '1':
            case '2':
            case '3':
            case '4':
            case '5':
            case '6':
            case '7':
            case '8':
            case '9':
                atmqueue.send(digit_pressed(c));
                break;
            case 'b':
                atmqueue.send(balance_pressed());
                break;
            case 'w':
                atmqueue.send(withdraw_pressed(50));
                break;
            case 'c':
                atmqueue.send(cancel_pressed());
                break;
            case 'q':
                quit_pressed=true;
                break;
            case 'i':
                atmqueue.send(card_inserted("acc1234"));
                break;
        }
    }
    bank.done();
    machine.done();
    interface hardware.done();
    atm_thread.join();
    bank_thread.join();
    if_thread.join();
}

```